

VoiceBOT — Technical Documentation

AI-Powered Multilingual FAQ Assistant with RAG Pipeline

1. Problem Statement

Context

Organizations with large employee bases face a recurring challenge: support teams (HR, IT help desk) spend a significant portion of their time answering the same repetitive questions — password resets, leave policies, payslip downloads, and profile updates.

This leads to:

- High support ticket volume — 60–70% of tickets are FAQ-type queries with documented answers.
- Slow response times — employees wait hours for answers already in policy documents.
- No voice-based self-service option for quick answers.
- Language barrier — Hindi-speaking employees lack support in their native language.

Problem Definition

Build an intelligent, voice-enabled FAQ assistant that:

- Answers questions strictly from uploaded company documents (no hallucination).
- Supports both Hindi and English via voice and text.
- Auto-detects the user's language and responds in the same language.
- Provides source citations (document name + page number) with every answer.
- Tracks real-time usage analytics derived from actual conversation data.
- Works with natural conversational language, not just exact keyword matches.

Target Users

- Employees: Seeking quick answers to HR/IT FAQs.
- Administrators: Uploading knowledge base documents.
- Management: Monitoring usage patterns via the analytics dashboard.

2. Design Approach

2.1 Architecture Overview

The system follows a layered service-oriented architecture with clear separation of concerns:

Frontend (React): Chat Window, Upload, Analytics, Settings

Backend (FastAPI): API routes, Services layer, Database layer

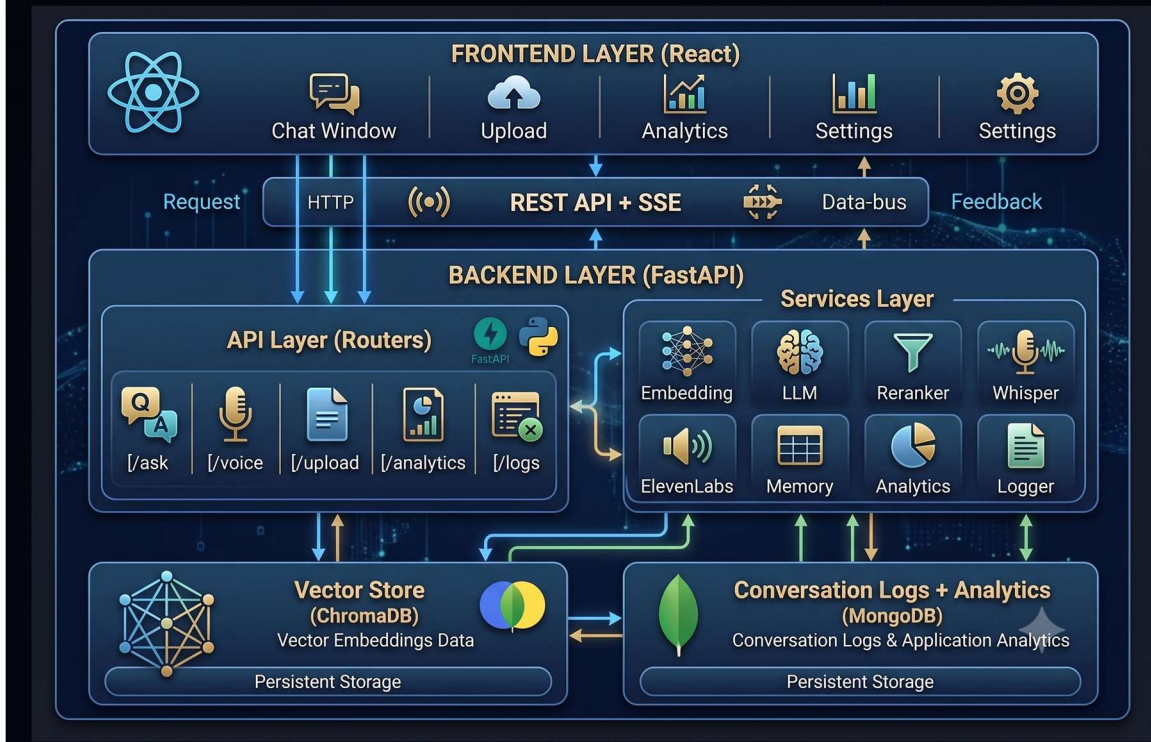
ChromaDB: Vector storage for document embeddings

MongoDB Atlas: Conversation logs and analytics data

External APIs: OpenAI (LLM + Whisper), ElevenLabs (TTS)

2.1 Architecture Overview

The system follows a layered service-oriented architecture with clear separation of concerns:



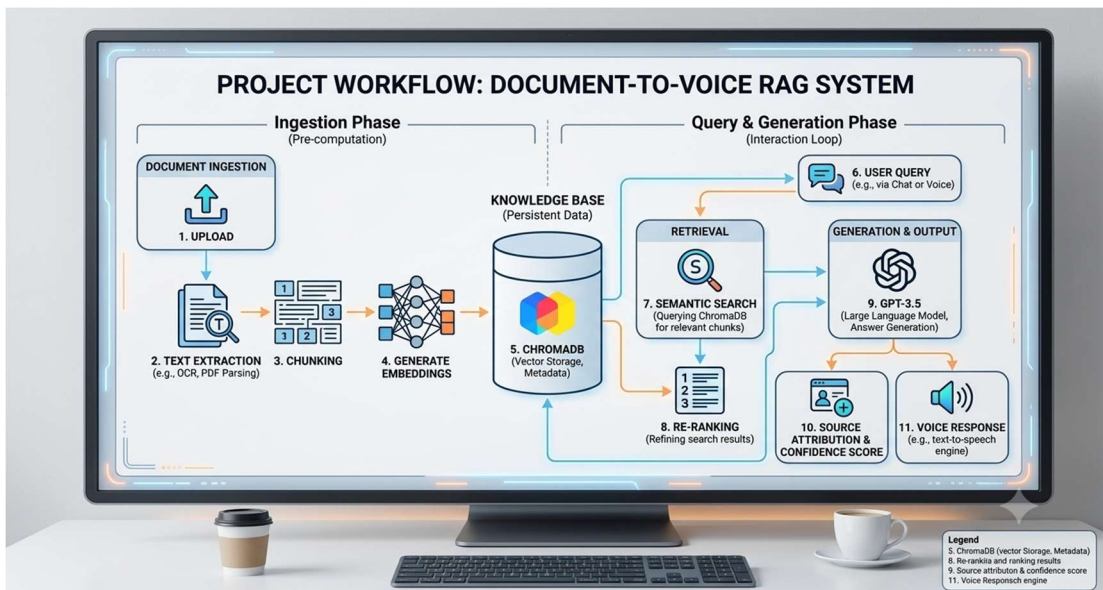
2.2 RAG Pipeline

Document Ingestion:

File Upload → FileParser (PDF/DOCX/TXT/CSV) → TextSplitter (600-char chunks, 100-char overlap) → EmbeddingService (768-dim vectors) → ChromaDB (persistent with source metadata)

Query Processing:

Question → EmbeddingService → ChromaDB cosine similarity (top 5) → Cross Encoder re-ranking → GPT-3.5 → Grounded answer + source citation + confidence score



2.3 Multilingual Design

Embedding: paraphrase-multilingual-mpnet-base-v2

STT: Whisper (English/Hindi)

LLM: Language-specific prompts

TTS: ElevenLabs (primary), gTTS (fallback)

Language Detection:

- Explicit user selection if available.
- Whisper auto-detection for voice.
- Unicode Devanagari ratio for text.

2.4 Voice Pipeline

User Speech → MediaRecorder → Web Audio API → Auto-stop on silence → Backend → Whisper STT → RAG → Answer → ElevenLabs/gTTS → Browser Playback

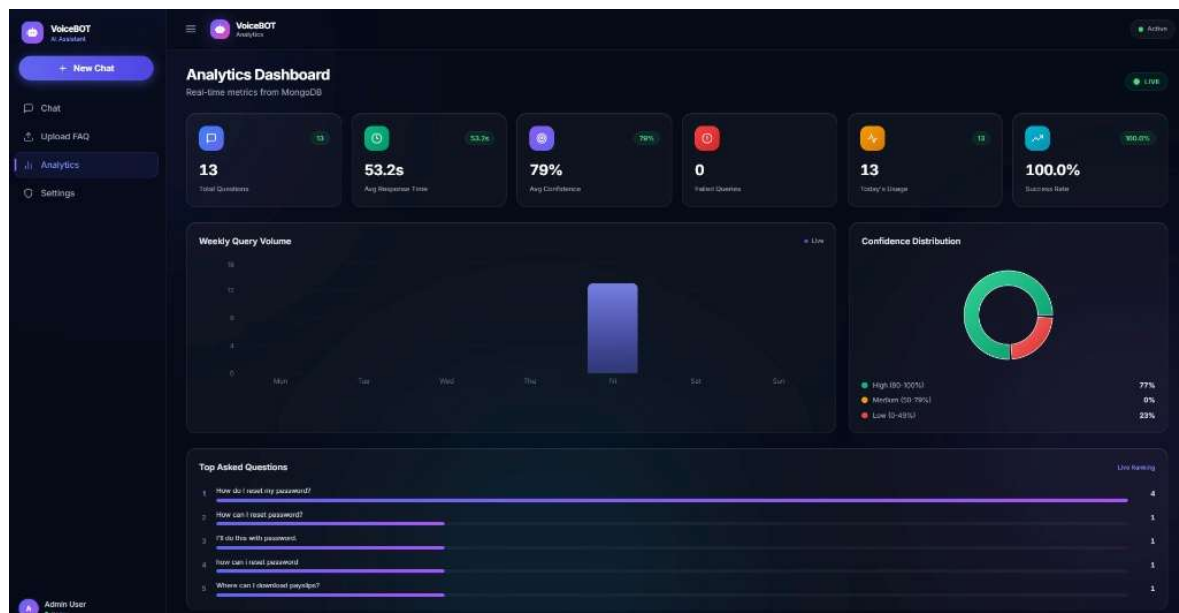
2.5 Real-Time Analytics

Flow:

User Question → MongoDB Log → SSE Notification → Analytics Aggregation → React Dashboard Update

Semantic Clustering:

Questions embedded using multilingual model and clustered with cosine similarity threshold 0.75.



2.6 Key Design Decisions

- Lazy model initialization.
- Graceful degradation for external service failures.
- In-memory sessions with MongoDB persistence.
- Cross-encoder re-ranking.
- SSE instead of WebSockets.
- 600-character chunks with 100 overlap.

3. APIs Used

External APIs:

- OpenAI Chat Completions (gpt-3.5-turbo)
- OpenAI Whisper (whisper-1)
- ElevenLabs TTS

Internal REST APIs:

POST /api/ask

POST /api/voice/ask

POST /api/voice/transcribe

POST /api/voice/synthesize

POST /api/upload/upload-faq

GET /api/analytics

GET /api/analytics/stream

GET /api/logs

GET /api/logs/{session_id}

GET /api/health

ML Models:

- paraphrase-multilingual-mpnet-base-v2 (768 dimensions)
- cross-encoder/ms-marco-MiniLM-L-6-v2

Database Schemas:

MongoDB: conversation_logs

Fields: session_id, question, answer, confidence, source, page, timestamp

ChromaDB:

id, embedding, document, metadata

4. Challenges Faced

1. Whisper translating Hindi speech to English.
2. Embedding model mismatch.
3. Cold start (~90 seconds).
4. MongoDB connection caching failure.
5. Duplicate analytics questions.
6. Circular imports.
7. IST timezone aggregation.

5. Future Improvements

Short-term:

- Persistent sessions
- JWT authentication
- Rate limiting
- Feedback system
- Accurate response timing

Medium-term:

- More Indian languages
- Document versioning
- Hybrid search
- Streaming responses
- PDF export

Long-term:

- AWS deployment
- Fine-tuned embedding models
- Multi-tenant architecture
- HRMS integration
- Mobile SDK
- Offline mode